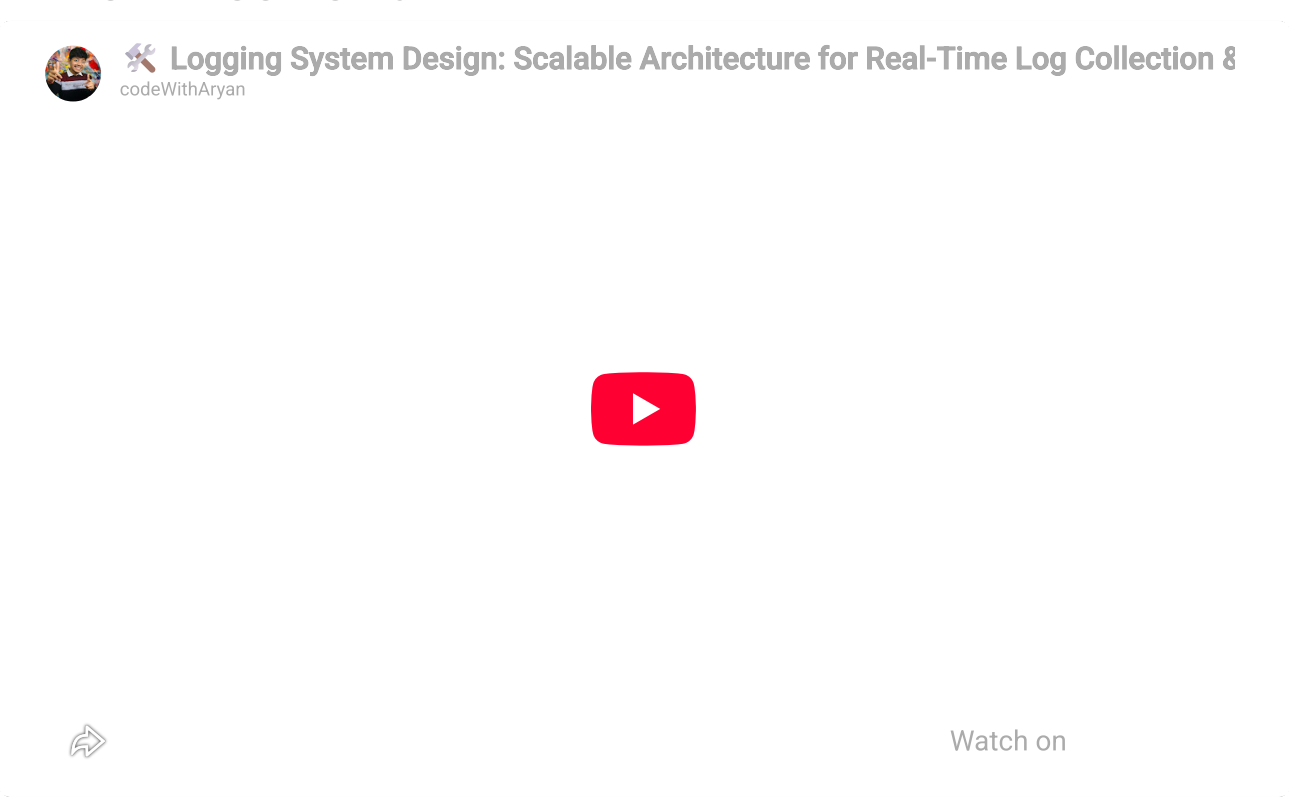
Design Logging System


🛠️ Logging System Design: Scalable Architecture for Real-Time Log Collection &
codeWithAryan
Watch on

Topic Tags:

system design | lld

🐈 **Github Codes Link**: https://github.com/aryan-0077/CWA-LowLevelDesignCode

Low-Level Design: Logging System 📝

A Logging System is designed to efficiently capture, filter, and output application events with varying levels of severity. The system needs to support multiple log levels, formatting options, destinations, and filtering capabilities while maintaining a clean separation of concerns. The system should be extensible, allowing for new log handlers, formatters, and destinations to be easily integrated.

Rules of the System:

Setup:
- The system supports multiple log levels (DEBUG, INFO, WARNING, ERROR, FATAL)
- Logs have attributes like timestamp, level, message, and context data
- The system can route logs to different destinations based on level and content

Operation:
- Applications create log messages with a specified severity level
- Each log passes through a chain of handlers that decide whether to process it
- Handlers can filter, format, and dispatch logs to various destinations
- The system can be configured to customize logging behavior at runtime

Features:
- Level-based filtering to control which logs are processed
- Formatting options for different output destinations
- Support for multiple output destinations (console, file, network, etc.)
- Context enrichment to add additional information to log entries

Interview Setting 🤝

Point 1: Introduction and Vague Problem Statement:

Interviewer: Let's start with a basic problem statement. Design a Logging System.

Candidate: Certainly! Here's my understanding of the Logging System requirements:
- The system will process log messages with different severity levels
- Logs should be filtered based on configuration settings
- The system should support various output destinations

Is this the expected direction for the design?

Interviewer: Yes, you are on the right track. Please continue.

Candidate: Great! Before diving into the design, I'd like to clarify a few requirements:
- What log levels should the system support?
- What types of destinations should we consider?
- Are there any specific filtering or formatting requirements?

Point 2: Clarifying Requirements:

Interviewer: We want a system that:
- Supports the standard log levels (DEBUG, INFO, WARNING, ERROR, FATAL)
- Log messages with timestamp, level, and content
- Multiple output destinations (console, file, database)
- Configuration mechanism for log level and output destination
- Extensibility for new log levels and output destinations

Candidate: To summarize, the key requirements are:
- A flexible logging system with standard severity levels
- Support for multiple output destinations
- Configurable filtering capabilities

Interviewer: Perfect, let's proceed.

Point 3: Identifying Key Components:

Candidate: Now that we have the requirements, let's identify the key components of our Logging System:
- Log Level: Enum representing different severity levels
- Log Message : Class which contains log message with all its attributes.
- Log Config : This class holds the configuration settings for the logger.
- Log Appender : Interface responsible for writing logs to destinations.
- Logger : Singleton Class that provided the main logging functionality.

Interviewer: That sounds good. Let's proceed with the design details.

Point 4: Design Challenges:

Interviewer: What design challenges do you anticipate?

Candidate: The key challenges for the Logging System include:
- Chain Construction: Building a flexible chain of handlers that can be modified at runtime
- Performance Considerations: Ensuring efficient log processing, especially for high-volume applications
- Configuration Management: Providing an easy way to configure the system

Point 5: Approach:

Interviewer: How would you approach these challenges?

Candidate: I propose using several design patterns effectively. Here are my strategies:
Chain of Responsibility Pattern for Log Processing:
- Creates a chain of log handlers
- Each handler decides whether to process the log or pass it along
- Enables separation of concerns for filtering, formatting, and output

Strategy Pattern for Log Appenders:
- Encapsulates different Log Appending Strategies.
- Allows runtime selection of Log appenders (Console, File etc.)
- Makes it easy to add new Appender options

Singleton Pattern for Logger Instance:
- Provides a global access point to the logging system
- Ensures consistent configuration across the application

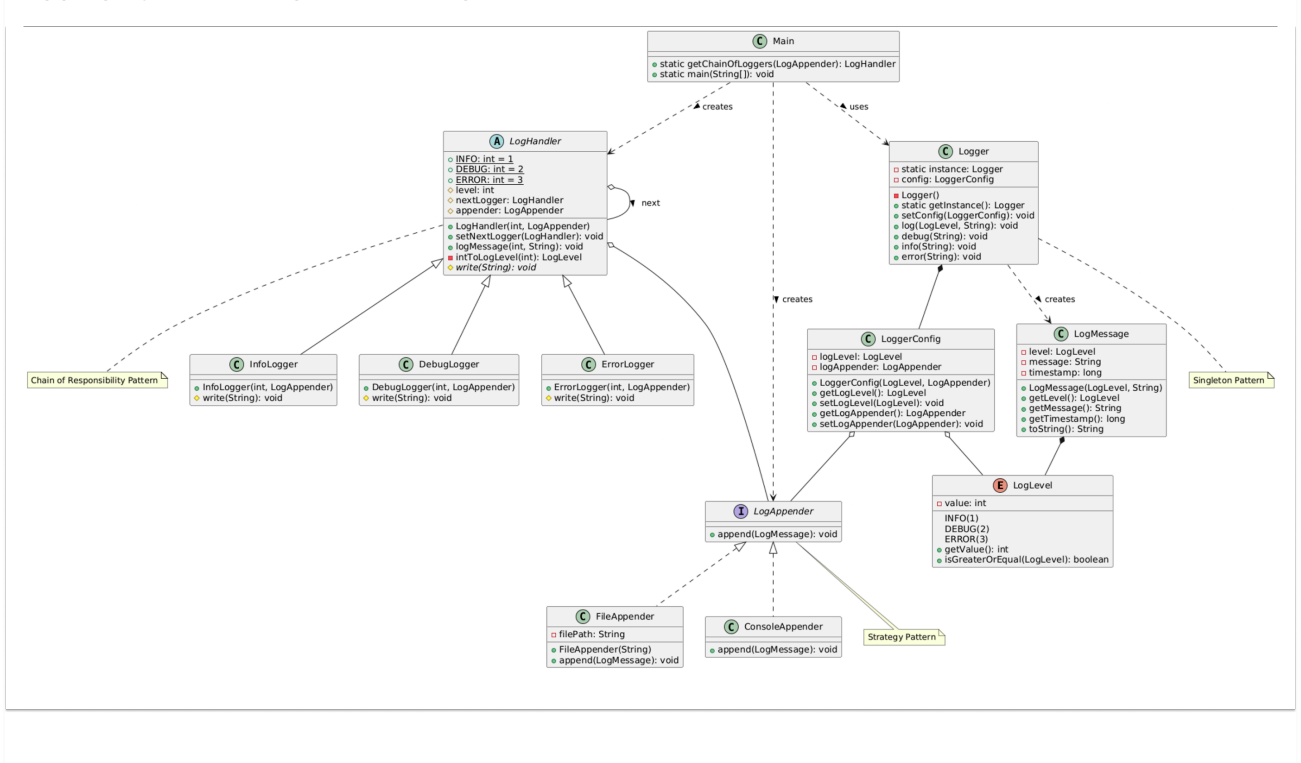
Interviewer: That sounds like a solid approach. Let's delve into the implementation details.

Point 6: Implementation:

Interviewer: Ready to discuss implementation?

Candidate: Yes. I'll focus on implementing the design patterns we discussed and show how they work together in the Logging System:

Logging System Design With Design Patterns :



1. Implementing the Core Classes:
Log Level Enum: Represents different logging severity levels (DEBUG, INFO, ERROR) and provides methods for comparison.

Java

```java
// Enum representing different log severity levels
public enum LogLevel {
    DEBUG(1),   // Debug-level messages (least severe)
    INFO(2),    // Informational messages
    ERROR(3)   // Error messages indicating failures

    // Numeric value associated with each log level
    private final int value;
    LogLevel(int value) {
        this.value = value;
    }

    // Getter method to retrieve the numeric value of a log level
    public int getValue() {
        return value;
    }

    // Method to compare log levels based on severity
    public boolean isGreaterOrEqual(LogLevel other) {
        return this.value >= other.value;
    }
}
```

- Log Message Class: Encapsulates a log entry with attributes like level, message, and timestamp, providing a formatted string representation.

Java

```java
public class LogMessage {
  // Log level of the message (e.g., INFO, DEBUG, ERROR)
  private final LogLevel level;
  // The actual log message content
  private final String message;
  // Timestamp when the log message was created
  private final long timestamp;
  // Constructor to initialize log level and message, setting the timestamp to current time
  public LogMessage(LogLevel level, String message) {
    this.level = level;
    this.message = message;
    this.timestamp = System.currentTimeMillis();
  }

  // Returns the log level of the message
  public LogLevel getLevel() {
    return level;
  }

  // Returns the log message content
  public String getMessage() {
    return message;
  }

  // Returns the timestamp of the log creation
  public long getTimestamp() {
    return timestamp;
  }

  // Formats the log message as a string with level, timestamp, and message
  @Override
  public String toString() {
    return "[" + level + "] " + timestamp + " - " + message;
  }
}
```

2. Chain of Responsibility Pattern for Log Handlers:
Log Handler Abstract Class: Serves as the base for log handlers, defining the chain behavior for passing log messages.

Java

```java
// Abstract logger defining the chain behavior
abstract class LogHandler {
  public static final int INFO = 1;
  public static final int DEBUG = 2;
  public static final int ERROR = 3;
  protected int level;
  protected LogHandler nextLogger;
  protected LogAppender appender; // the appender where we need to append the logs
  // Constructor to initialize with appender
  public LogHandler(int level, LogAppender appender) {
    this.level = level;
    this.appender = appender;
  }

  // Set the next logger in the chain
  public void setNextLogger(LogHandler nextLogger) {
    this.nextLogger = nextLogger;
  }

  // Corrected to use LogLevel instead of int for consistency
  public void logMessage(int level, String message) {
    if (this.level >= level) {
      // Convert int level to LogLevel enum
      LogLevel logLevel = intToLogLevel(level);
      LogMessage logMsg = new LogMessage(logLevel, message);
      // Use the appender to log
      if (appender != null)
        appender.append(logMsg);
      write(message);
    }
    else if (nextLogger != null)
      nextLogger.logMessage(level, message);
  }

  // Helper method to convert int level to LogLevel enum
  private LogLevel intToLogLevel(int level) {
    switch (level) {
      case INFO:
        return LogLevel.INFO;
      case DEBUG:
        return LogLevel.DEBUG;
      case ERROR:
        return LogLevel.ERROR;
      default:
        return LogLevel.INFO;
    }
  }

  // Each concrete logger will implement its own writing mechanism
  abstract protected void write(String message);
}
```

- Concrete Log Handler Classes: Implements INFO, DEBUG, and ERROR handlers for filtering and processing log messages based on severity.

Info Logger :

Java

```java
// Logger for INFO level messages
class InfoLogger extends LogHandler {
  public InfoLogger(int level, LogAppender appender) {
    super(level, appender);
  }

  @Override
  protected void write(String message) {
    System.out.println("INFO: " + message);
  }
}
```

- Debug Logger :

Java

```java
// Logger for DEBUG level messages
    class DebugLogger extends LogHandler {
        public DebugLogger(int level, LogAppender appender) {
            super(level, appender);
        }

        @Override
        protected void write(String message) {
            System.out.println("DEBUG: " + message);
        }
}
```

- Error Logger :

Java

```java
// Logger for ERROR level messages
    class ErrorLogger extends LogHandler {
        public ErrorLogger(int level, LogAppender appender) {
            super(level, appender);
        }

        @Override
        protected void write(String message) {
            System.out.println("ERROR: " + message);
        }
}
```

3. Strategy Pattern for Log Appenders:
Log Appender Interface: Defines a contract for appending log messages.

Java

```java
public interface LogAppender {
        void append(LogMessage logMessage);
}
```

Concrete Log Appenders :
a. File Appender Class :

Java

```java
// Appends log messages to a file
public class FileAppender implements LogAppender {
  private final String filePath; // Path to the log file

  // Constructor to set the file path
  public FileAppender(String filePath) {
    this.filePath = filePath;
  }

  // Appends a log message to the file
  @Override
  public void append(LogMessage logMessage) {
    try (FileWriter writer = new FileWriter(filePath, true)) {
      writer.write(logMessage.toString() + "
"); // Write log to file
    } catch (IOException e) {
      e.printStackTrace(); // Print error if file writing fails
    }
  }
}
```

b. Console Appender Class :

Java

```java
// Prints log messages to the console
    public class ConsoleAppender implements LogAppender {
        // Appends a log message to the console
        @Override
        public void append(LogMessage logMessage) {
            System.out.println(logMessage); // Print log to console
        }
}
```

4. **Implement the Logger Config file :**
Logger Config: Manages configuration details like log level and appender, making the logger flexible and configurable.

Java

```java
public class LoggerConfig {
  private LogLevel logLevel;
  private LogAppender logAppender;
  public LoggerConfig(LogLevel logLevel, LogAppender logAppender) {
    this.logLevel = logLevel;
    this.logAppender = logAppender;
  }

  public LogLevel getLogLevel() {
    return logLevel;
  }

  public void setLogLevel(LogLevel logLevel) {
    this.logLevel = logLevel;
  }

  public LogAppender getLogAppender() {
    return logAppender;
  }

  public void setLogAppender(LogAppender logAppender) {
    this.logAppender = logAppender;
  }
}
```

5. **Singleton Pattern for Main Logger Class :**
Logger: Ensures a single, globally accessible instance of the logger, supporting thread-safe configuration updates.

Java

```java
public class Logger {
  private static final ConcurrentHashMap<String, Logger> instances = new ConcurrentHashMap<>
();
  private LoggerConfig config;
  // Private constructor to enforce singleton pattern
  private Logger(LogLevel logLevel, LogAppender logAppender) {
    config = new LoggerConfig(logLevel, logAppender);
  }

  // Get instance based on LogLevel and LogAppender
  public static Logger getInstance(LogLevel logLevel, LogAppender logAppender) {
    String key = logLevel.name() + "_" + logAppender.getClass().getName();
    // Compute instance if absent (thread-safe lazy initialization)
    return instances.computeIfAbsent(key, k -> new Logger(logLevel, logAppender));
  }

  // Updates the logger configuration
  public void setConfig(LoggerConfig config) {
    synchronized (Logger.class) { // Ensure thread safety while updating config
      this.config = config;
    }
  }

  // Logs a message if the level meets the configured threshold
  public void log(LogLevel level, String message) {
    if (level.getValue() >= config.getLogLevel().getValue()) {
      LogMessage logMessage = new LogMessage(level, message);
      config.getLogAppender().append(logMessage);
    }
  }

  public void debug(String message) {
    log(LogLevel.DEBUG, message);
  }

  public void info(String message) {
    log(LogLevel.INFO, message);
  }

  public void error(String message) {
    log(LogLevel.ERROR, message);
  }
}
```

6. **Main Method / Client Code to Run the Logging System :**
Demonstrates the logging system by initializing loggers, selecting appenders, and processing log messages through the chain of responsibility.

Java

```java
// Client class to demonstrate logging system
public class Main {
  // Build the chain of loggers: INFO -> DEBUG -> ERROR
  private static LogHandler getChainOfLoggers(LogAppender appender) {
    LogHandler errorLogger = new ErrorLogger(LogHandler.ERROR, appender);
    LogHandler debugLogger = new DebugLogger(LogHandler.DEBUG, appender);
    LogHandler infoLogger = new InfoLogger(LogHandler.INFO, appender);
    infoLogger.setNextLogger(debugLogger);
    debugLogger.setNextLogger(errorLogger);
    return infoLogger;
  }

  public static void main(String[] args) {
    // Select the log appender (console or file)
    LogAppender consoleAppender = new ConsoleAppender();
    LogAppender fileAppender = new FileAppender("logs.txt");
    // Create the chain of loggers with the console appender
    LogHandler loggerChain = getChainOfLoggers(consoleAppender);

    // Use a single logging approach to avoid duplication
    System.out.println("Logging INFO level message:");
    loggerChain.logMessage(LogHandler.INFO, "This is an information.");
    System.out.println("
Logging DEBUG level message:");
    loggerChain.logMessage(LogHandler.DEBUG, "This is a debug level information.");
    System.out.println("
Logging ERROR level message:");
    loggerChain.logMessage(LogHandler.ERROR, "This is an error information.");

    // Demonstrate the singleton Logger usage as an alternative
    System.out.println("
Using Singleton Logger:");
    Logger logger = Logger.getInstance(LogLevel.INFO, consoleAppender);
    logger.setConfig(new LoggerConfig(LogLevel.INFO, fileAppender));
    logger.error("Using singleton Logger - Error message");
  }
}
```

Interviewer: Looks good. What makes your approach effective?

Candidate: Here are the key strengths of my approach for the Logging System:
1. Flexible Chain of Responsibility:
The design implements the Chain of Responsibility pattern to create a flexible pipeline of log handlers. Each handler has a specific purpose (filtering, formatting, output), and they can be composed in any order.

2. Single Responsibility Principle:
Each class has a clear, focused responsibility. Formatters handle formatting, filters handle filtering, and appenders handle output destinations. This makes the code easier to understand and maintain.

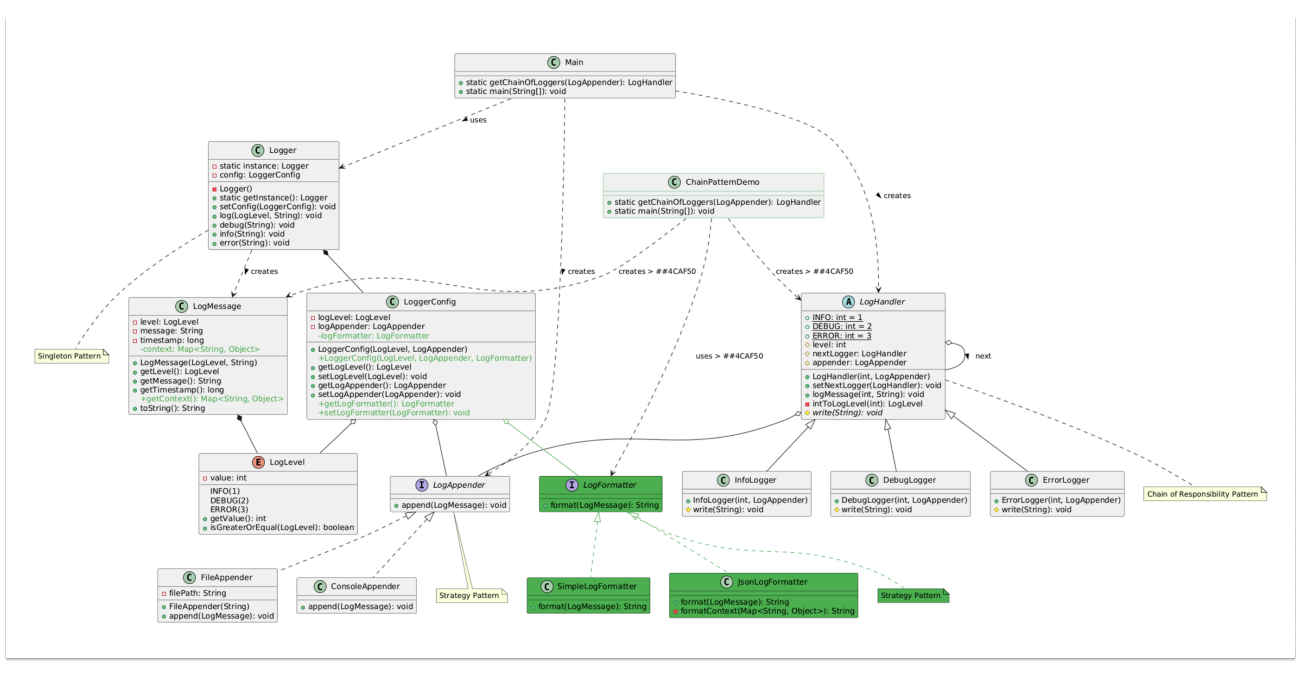
3. Open/Closed Principle:
The system is open for extension but closed for modification. New handlers, formatters, or appenders can be added without changing existing code.

4. Configurability:
The Log Manager allows for runtime configuration of the handler chain, making it easy to adapt the logging behavior to different environments or requirements.

5. Separation of Concerns:
The design cleanly separates different concerns such as log creation, filtering, formatting, and output.

Extensibility :

1. Include Support for Log formatting :



Create Log formatter Implementation Using Strategy Pattern :
Log Formatter Interface: Defines the contract for log formatters.

Java

```java
// Interface for log formatters
    public interface LogFormatter {
        String format(LogMessage logEntry);
    }
```

- Simple Log Formatter: A basic formatter that outputs logs in a simple format.

Java

```java
// A simple log formatter
    public class SimpleLogFormatter implements LogFormatter {
        @Override
        public String format(LogMessage logEntry) {
            return String.format("[%s] %s: %s",
                new Date(logEntry.getTimestamp()),
                logEntry.getLevel(),
                logEntry.getMessage());
        }
}
```

- Json Log Formatter: A formatter that outputs logs in JSON format.

Java

```java
import java.util.Date;
import java.util.Map;
// A JSON log formatter that converts log entries into JSON format
public class JsonLogFormatter implements LogFormatter {
  // Formats a LogEntry into a structured JSON string
  @Override
  public String format(LogMessage logEntry) {
    StringBuilder sb = new StringBuilder();
    sb.append("{
"); // Start of JSON object
    sb.append("  \"timestamp\": \"")
        .append(new Date(logEntry.getTimestamp()))
        .append("\",
"); // Format timestamp
    sb.append("  \"level\": \"").append(logEntry.getLevel()).append("\",
"); // Log level
    sb.append("  \"message\": \"").append(logEntry.getMessage()).append("\",
"); // Log message
    sb.append("  \"context\": ")
        .append(formatContext(logEntry.getContext()))
        .append("
"); // Format context
    sb.append("}"); // End of JSON object
    return sb.toString();
  }

  // Helper method to format the context map into a JSON-like structure
  private String formatContext(Map<String, Object> context) {
    if (context.isEmpty()) {
      return "{}"; // Return empty JSON object if context is empty
    }
    StringBuilder sb = new StringBuilder();
    sb.append("{
");
    boolean isFirstEntry = true;
    for (Map.Entry<String, Object> entry : context.entrySet()) {
      if (!isFirstEntry) {
        sb.append(",
"); // Add a comma before each new entry (except the first one)
      }
      isFirstEntry = false;
      sb.append("    \"").append(entry.getKey()).append("\": "); // Key formatting
      Object value = entry.getValue();
      if (value instanceof String) {
        sb.append("\"").append(value).append("\""); // Wrap string values in quotes
      } else {
        sb.append(value); // Directly append non-string values
      }
    }
    sb.append("
  }"); // End of JSON object for context
    return sb.toString();
  }
}
```

- Modify the Client code to handle Formatting Logic :

Java

```java
public class ChainPatternDemo {
  // Build the chain of loggers: INFO -> DEBUG -> ERROR
  private static LogHandler getChainOfLoggers(LogAppender appender) {
    LogHandler errorLogger = new ErrorLogger(LogHandler.ERROR, appender);
    LogHandler debugLogger = new DebugLogger(LogHandler.DEBUG, appender);
    LogHandler infoLogger = new InfoLogger(LogHandler.INFO, appender);
    infoLogger.setNextLogger(debugLogger);
    debugLogger.setNextLogger(errorLogger);
    return infoLogger;
  }

  public static void main(String[] args) {
    // Select the log appender (console or file)
    LogAppender consoleAppender = new ConsoleAppender();
    LogAppender fileAppender = new FileAppender("logs.txt");
    // can also create a factory for formatters instead of directly hard coding the type of
    // formatter
    LogFormatter formatter = new JsonLogFormatter(); // Apply JSON formatting
    // Choose the appender (change this to fileAppender to write logs to a file)
    LogHandler loggerChain = getChainOfLoggers(consoleAppender);
    System.out.println("Logging INFO level message:");
    LogMessage infoMessage = new LogMessage(LogLevel.INFO, "This is an information.");
                loggerChain.logMessage(LogHandler.INFO,
formatter.format(infoMessage.getMessage());
            System.out.println("
Logging DEBUG level message:");
            LogMessage debugMessage = new LogMessage(LogLevel.DEBUG, "This is a debug level
information.");
            loggerChain.logMessage(LogHandler.DEBUG,
formatter.format(debugMessage.getMessage()));
            System.out.println("
Logging ERROR level message:");
            LogMessage errorMessage = new LogMessage(LogLevel.ERROR, "This is an error
information.");
            loggerChain.logMessage(LogHandler.ERROR,
formatter.format(errorMessage.getMessage()));
  }
}
```

✨ Conclusion:

This low-level design for a Logging System demonstrates a well-structured, scalable, and extensible architecture. It follows the Chain of Responsibility pattern to manage log processing, allowing for a flexible and configurable system that can adapt to various requirements.

The design incorporates several other patterns to address specific concerns:
- Strategy Pattern for Different Log Appenders.
- Chain of Responsibility Pattern for Shifting Between different types of Concrete Loggers.
- Singleton Pattern for global access.

The system supports various features including level-based filtering, content filtering, multiple output destinations. It's designed to be thread-safe, extensible, and configurable at runtime.

In an interview setting, presenting this design highlights your ability to apply design patterns appropriately to solve real-world problems while maintaining principles of clean, maintainable code. It also demonstrates your understanding of logging systems' requirements and challenges.